

Crucigrama CBD

Pedro González Marcos

27 de Mayo 2024

Índice

1. Introducción	2
2. Crucigrama	2
2.1. Cuadrícula	3
2.2. Pistas	3
3. Modelo de datos	5
3.1. Cuadrícula	5
3.2. Celdas	5
3.3. Pistas	7
3.4. Crucigrama	8
3.5. Algoritmo	10
4. Marco de trabajo	10
5. Tecnologías	13
5.1. Express	13
5.1.1. Listar crucigramas	14
5.1.2. Obtener detalles de un crucigrama	14
5.1.3. Comprobar solución del crucigrama	15
5.1.4. Buscar palabras en el diccionario	16
5.2. MongoDB	17
5.3. Elasticsearch	17
6. Conclusiones	20
7. Anexo	20
7.1. Prerequisitos	20
7.2. Script	20

1. Introducción

La idea de crear una aplicación web de crucigramas surgió mientras estaba preparando la teoría de CBD. En los temas había muchas palabras y conceptos que podían sintetizarse perfectamente en una definición. Quería hacer una herramienta que me fuese útil para estudiar y que sirviese como proyecto para la asignatura.

El stack tecnológico Mongo y Elasticsearch era una buena elección para implementar este pasatiempo. Los crucigramas se pueden representar en un simple documento. La cuadrícula es una lista de listas que no soportan las BBDD relacionales.

En el trabajo se han cubierto los siguientes puntos:

- Modelar el dominio del problema a un documento en MongoDB
- Explorar los tipos de datos que MongoDB ofrece listas, objetos, propiedades sin definir...
- Explorar el motor de búsqueda de Elasticsearch
- Hacer una aplicación web de un crucigrama que un usuario pueda jugar

De todos esos puntos se han implementado con éxito, la interfaz está hecha en React y la API está hecha en Express, el modelo de datos exprime al máximo las ventajas de los documentos de Mongo y se ha implementado un buscador de palabras en un diccionario utilizando Elasticsearch.

2. Crucigrama

Para poner en contexto al que no esté familiarizado con este juego. El crucigrama es un pasatiempo donde tienes que introducir palabras en una cuadrícula siguiendo las pistas proporcionadas en horizontal y en vertical por el crucigramista.

Aunque no te sepas todas las palabras puedes llegar a resolver el crucigrama rellenando otras pistas. Puede que al principio sea complicado y consultes muchas palabras, pero conforme vas haciendo más pasatiempos te vuelves más experto.

Por este motivo es un crucigrama divertido, te expones diariamente a un reto intelectual, una oportunidad de aprender palabras que desconocías.

De hecho, en Estados Unidos publican diariamente en el periódico 1 crucigrama. Los del lunes son los más sencillos y son pequeños 11 x 11, los

domingos son muy complicados siendo más grandes 21 x 21 y con pistas más crípticas.

Hay todo un gremio especializado en hacer crucigramas, ya que hacer un buen crucigrama es muy complicado. Hay que tener en cuenta varios factores como la simetría de la cuadrícula, la calidad de las pistas, el tema del crucigrama.

En la dos secciones posteriores vamos a hablar de los dos elementos elementales que tiene que tener un crucigrama:

- Cuadrícula
- Pistas

2.1. Cuadrícula

La cuadrícula es donde podemos introducir la solución del crucigrama. Distinguimos dos tipos de celdas:

- Las celdas negras, bloques...
- Las celdas blancas

En las celdas negras no se puede escribir nada y sirven para delimitar la longitud de las palabras. Los crucigramistas, suelen jugar con los bloques para crear sus crucigramas. Pueden ponerlas por conveniencia para separar palabras o por simple estética. Dependiendo donde esten situadas, el crucigrama tendrá más o menos palabras.

No obstante, en las celdas blancas se introducen respuestas a las pistas. Además, si nos fijamos en el crucigrama de la figura 2, podemos ver que algunas celdas tienen numeración. Esto tiene que ver con la forma de dar pistas en el crucigrama.

Hay dos variantes para dar las pistas, enumerar la celdas como en la figura 2 y numerar las filas y las columnas de la figura 1.

2.2. Pistas

Un crucigrama está formado por palabras en vertical y en horizontal. Existen dos formas para hacer referencia a las pistas en la cuadrícula.

- Coordenadas: Se numeran las filas y las columnas del crucigrama. Por cada palabra que haya en una fila o en una columna se añade a la lista de pistas horizontales o verticales en el número que le corresponda. Por ejemplo en la figura 1 la tercera fila tiene 2 palabras, por lo que en las pistas horizontales el número 3 tiene 2 pistas separadas por puntos.

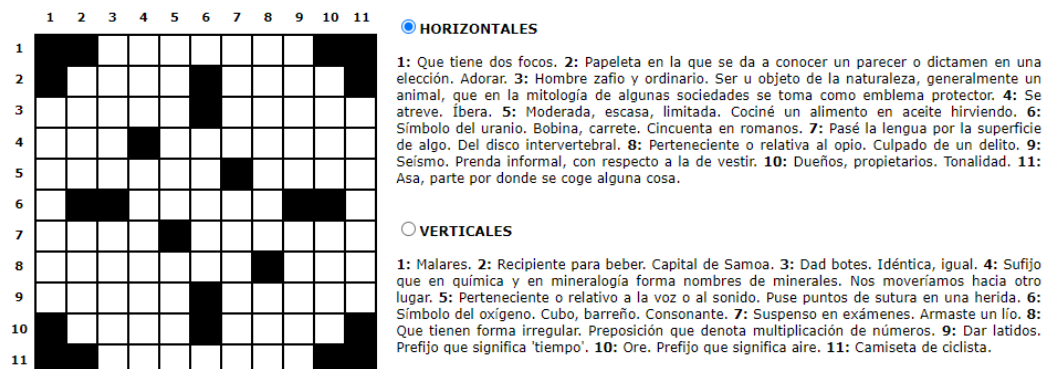


Figura 1: Crucigrama numerando filas y columnas

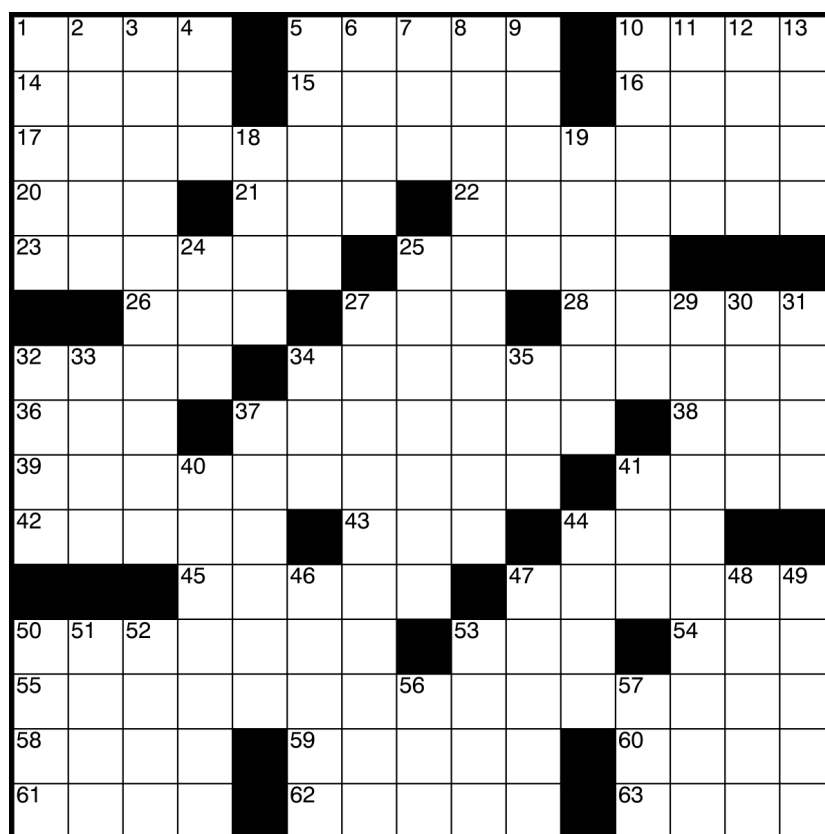


Figura 2: Crucigrama vacío numerado

- Numeración de casillas: Se numeran las celdas que empiecen una palabra. Puede ver más detalles de la numeración de las celdas en la sección [3.5](#).

3. Modelo de datos

Si analizamos detenidamente un crucigrama distinguimos 3 elementos básicos:

- La cuadrícula
- La solución
- Las pistas

3.1. Cuadrícula

Si observamos la figura [2](#) las celdas están dispuestas en filas y columnas.

Es sensato pensar en modelar este objeto con una estructura de datos muy utilizado en el software, una lista. Como tiene coordenadas vamos a utilizar una lista para guardar las celdas de una fila, por lo que habrá tantos elementos en la lista como filas haya en la cuadrícula.

Para representar las columnas de la cuadrícula utilizamos otra lista para almacenar las posiciones de la celda y sus atributos.

Una cuadrícula será válida si cada fila tiene el mismo número de celdas. En la figura [3](#) podemos ver la relación de un cuadrícula con sus celdas.

3.2. Celdas

Dentro de una cuadrícula nos encontramos como ya se ha comentado dos tipos de celdas blancas y negras. El tipo de dato adecuado para modelar este hecho es un atributo Booleano. Ahora hay que pensar en la semántica de esos dos estados, si este atributo es verdadero la celda será negra y en caso contrario será blanca.

En la figura [3](#) el atributo `dark` indica el estado de una celda.

Otro elemento crucial a representar de las celdas es su solución. Por simplicidad todas las celdas tendrá un carácter solución. Más tarde esto nos será útil para calcular la solución fácilmente si no hacemos distinciones de celdas.

En figura [3](#) en la clase *Cell* este atributo se llama `solution` y es de tipo cadena.

A estas alturas ya sabemos como representar la cuadrícula y podemos introducir las soluciones pero ¿Cómo vamos a resolver el crucigrama si no hay referencias a las pistas en la cuadrícula?.

A estas alturas tenemos que elegir la estrategia a implementar para hacer referencia a las pistas ya previamente explicada en la sección 2.2. La estrategia que he elegido ha sido la de numerar las casillas.

Es bastante intuitivo para los usuarios que resuelven el crucigrama. Cada celda numerada es una palabra y al contrario que la estrategia de coordenadas en el que una fila contiene una o más palabras separadas por puntos.

Con esta estrategia en mente es importante destacar que no todas las celdas están numeradas. Por ejemplo si nos fijamos en la figura 4, la celda de la tercera fila y la segunda columna no tiene ningún número asignado. Esto se debe a que esa celda ya pertenece a la palabra 1 en vertical y a la palabra 6 en horizontal.

Con todo esto en mente la etiqueta o número de una celda será un atributo opcional. En el modelo de datos de la figura 3 este atributo se llama `label`.

Estos serían los atributos básicos que tendría nuestro modelo. Ahora bien, si queremos que el usuario no se pierda mientras está resolviendo una pista, podemos iluminar las casillas que corresponden a la pista en la que está concentrado como podemos ver en la figura 4 del crucigrama del New York Times.

Con esta nueva característica tenemos que expandir el modelo y añadir más atributos.

Podemos modelar esto con dos atributos opcionales que tendrán una etiqueta de la palabra horizontal y vertical a la que pertenecen. Estos dos atributos opcionales no pueden ser simultáneamente nulos ya que una celda siempre pertenece al menos a una palabra del crucigrama ya sea en horizontal o en vertical. No obstante, puede darse la situación de que una celda no pueda pertenecer a una palabra en horizontal o viceversa.

Los atributos opcionales serán cadenas de texto, la primera parte de la cadena es el número de la palabra a la que pertenecen y la segunda parte de la cadena es una etiqueta que identifica si pertenece a una palabra en horizontal o en vertical, `<número><A/D>` (Horizontal en inglés es Across y Vertical en inglés es Down).

Estos dos atributos opcionales en el modelo de datos de la figura 3 se llaman `across` si la celda pertenece a una palabra en horizontal y `down` si la celda pertenece a una palabra en vertical.

La cuadrícula modelada de la figura 4 como un documento en JSON tendría el siguiente aspecto (Se ha modelado la primera fila de la cuadrícula por brevedad; La solución a la palabra 1 en horizontal es VIP):

```
[
  [
    {
      dark: true
      solution: "."
    },
    {
      dark: false,
      solution: "V",
      label: "1",
      across: "1A",
      down: "1D"
    },
    {
      dark: false,
      solution: "I",
      label: "2",
      across: "1A",
      down: "2D"
    },
    {
      dark: false,
      solution: "P",
      label: "3",
      across: "1A",
      down: "3D"
    },
  ],
  ...,
]
```

3.3. Pistas

Otra parte fundamental a modelar del crucigrama son las pistas. Como vemos en la figura 4, situada a la derecha de la cuadrícula podemos ver dos columnas con pistas en horizontal y en vertical (Across y Down).

Siguiendo esta misma filosofía, podemos pensar en una pista como un objeto con dos atributos que contengan una lista de las pistas en horizontal y en vertical.

En el modelo conceptual sería simplemente una lista con pistas sin clasificar, pero como en un documento tenemos la facilidad de añadir atributos

vamos a distinguir las pistas y no ponerlas en la misma lista.

Cada pista del crucigrama debe de tener dos atributos, la referencia de la pista en el crucigrama y el texto de la pista. Estos dos atributos nuevamente serán cadenas de texto.

En el modelo de datos de la figura 4 esto atributos se llaman `label` y `hint` respectivamente.

El documento que modela a las pistas del crucigrama de la figura 4 se tendría el siguiente aspecto:

```
{
  "across": [
    {
      "label": "1A",
      "hint": "Someone who might have a special line to the entrance"
    },
    ...
  ]
  "down": [
    {
      "label": "1D",
      "hint": "Physicist for whom an electrical measurement is named"
    },
    ...
  ]
}
```

3.4. Crucigrama

Con la cuadrícula, la solución y las pistas ya tenemos un crucigrama funcional que podemos representar en un documento.

Aunque antes de eso podemos añadirle más información y semántica al modelo.

Como los crucigramas se publican diariamente añadimos un atributo de tipo fecha. Además vamos a imponer como regla de negocio que no puede más de un crucigrama publicado en un día.

También añadimos al modelo 3 atributos derivados, el número de palabras horizontales, el número de palabras verticales y el número total de palabras que serán calculados.

Todos estos atributos están en la clase *Crossword* de la figura 3.

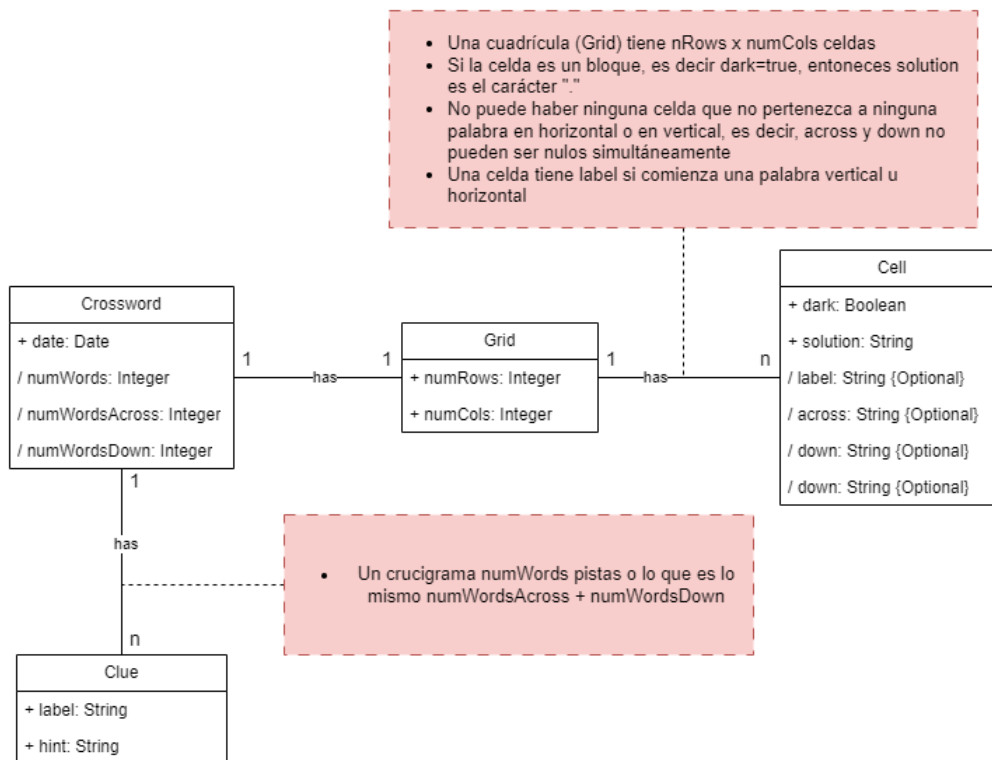


Figura 3: Modelo conceptual del crucigrama

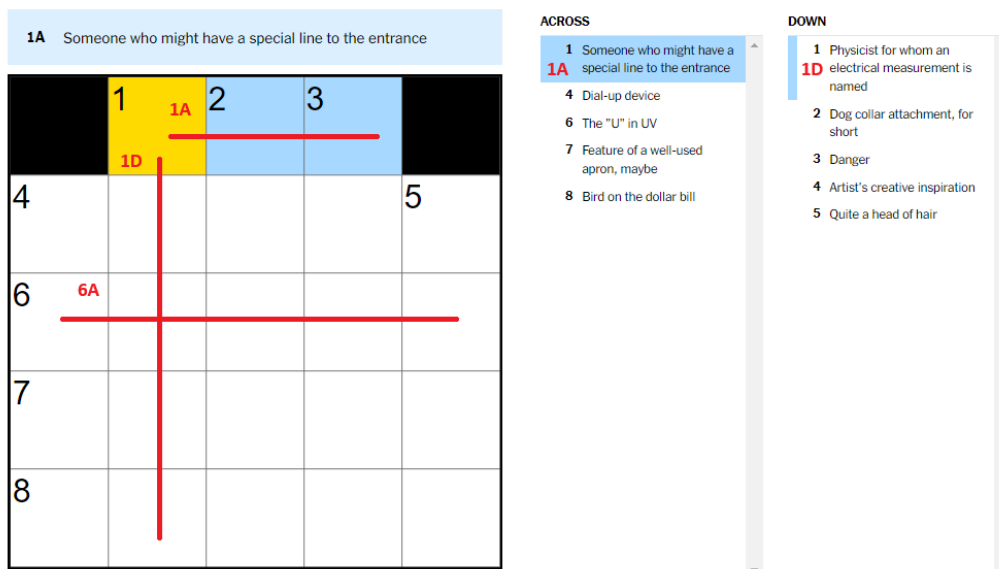


Figura 4: Crucigrama con celdas numeradas

El documento final del crucigrama de la figura 4 tendría el siguiente aspecto con todos los documentos ensamblados. Para no repetir los documentos anteriores se ha puesto una elipsis:

```
{
  "date": { "$date": "2024-06-24T00:00:00.000Z" },
  "numRows": 5,
  "numCols": 5,
  "numWords": 10,
  "numWordsAcross": 5,
  "numWordsDown": 5,
  "crossword": [...],...
  "clues": { across: [...], down: [...] }
}
```

3.5. Algoritmo

En esta sección describiremos brevemente el algoritmo utilizado para numerar y etiquetar las celdas del crucigrama.

Está pensado para trabajar con crucigramas americanos [1] que tienen una serie de reglas como:

- Las palabras al menos tienen 3 letras.
- El crucigrama es simétrico si lo rotamos 180 grados.

El algoritmo detectará una palabra, si al menos su longitud es de tres caracteres.

En la figura 5 se muestra el pseudocódigo [2] que describe al algoritmo:

4. Marco de trabajo

Con el modelo de datos terminado ya tenemos un documento que podemos pasar a EJJSON [6] (archivo compatible con el RFC 8259 en el que se especifican en JSON los tipos de datos de MongoDB).

Hay un problema con el modelo planteado, es bastante largo como para que alguien escriba el JSON manualmente.

Por esta razón se ha desarrollado el marco del proyecto un script que transforma plantilla de un crucigrama en un formato personalizado a un documento plantilla EJJSON (Paso 1 de la figura 6).

```

posicionesHorizontales = {}
posicionesVerticales = {}
numeroEtiqueta = 0

Procedimiento detectarEtiquetas(crucigrama)
  Para cada fila en crucigrama
    Para cada celda en fila
      Si celda es bloque Entonces
        Continua
      Fin Si

      Si (recorriendo el crucigrama en horizontal la
        celda anterior es bloque o está en primera columna
        y tiene longitud mínima)[a] o (recorriendo
        el crucigrama en vertical la celda anterior es
        bloque en vertical o está en primera fila y
        tiene longitud mínima)[b] Entonces
        nuevaEtiqueta <- nuevaEtiqueta + 1
      Fin Si

      Si [a] Entonces
        posicionesHorizontales <- etiquetaCelda(celda, nuevaEtiqueta)
      Fin Si

      Si [b] Entonces
        posicionesVerticales <- etiquetaCelda(celda, nuevaEtiqueta)
      Fin Si
    Fin Para
  Devuelve posicionesHorizontales, posicionesVerticales
Fin Procedimiento

```

Figura 5: Algoritmo detector de etiquetas

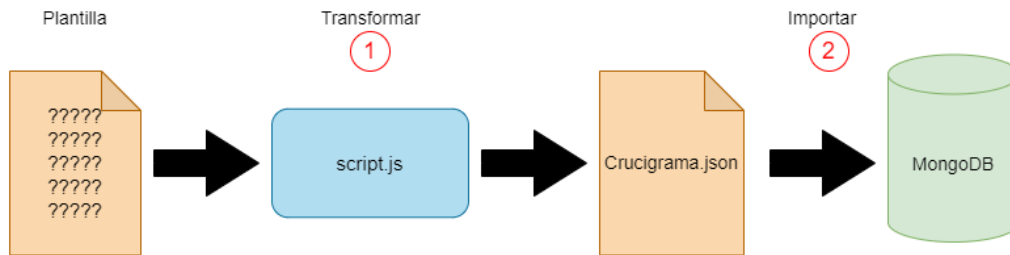


Figura 6: Transformación de la plantilla personalizada de la cuadrícula a una plantilla EJJSON que puede leer MongoDB

Posteriormente rellenaremos la plantilla con las pistas y finalmente importaremos el documento a nuestra BBDD (Paso 2 de la figura 6).

La plantilla personalizada para representar un crucigrama es el de la página *Words Up?* [5] que tiene un gran librería de crucigramas.

El formato propuesto por la página es el siguiente:

- Es un archivo de texto corriente.
- Las bloques se representan con un punto.
- Las celdas en blanco se representan con un interrogante.
- El número de filas del crucigrama es el número de líneas del archivo de texto.
- El número de columnas del crucigrama es la longitud de caracteres de una línea. La longitud debe ser consistente en todas las líneas del crucigrama. Es decir todas las líneas deben tener la misma longitud de caracteres.
- El saldo de línea `\n` o `\r\n` es la terminación de una fila y el comienzo de la siguiente.
- Cualquier carácter que no sea un punto, un interrogante o un salto de línea no cumple con el formato esperado del archivo de texto.
- No pueden haber líneas en blanco.

Por ejemplo en la figura 7 podemos ver el formato de archivo propuesto.

Este formato facilita la creación de plantillas, siendo la alternativa de escribir el EJJSON menos legible y mucho más verboso.

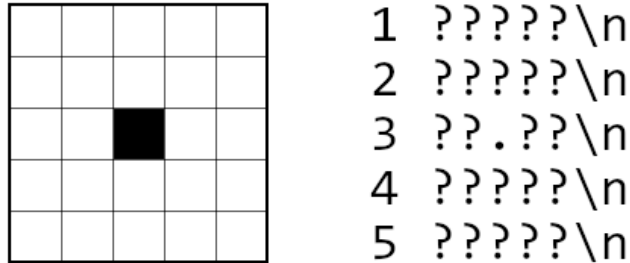


Figura 7: A la izquierda un crucigrama con 5 filas y 5 columnas. A la derecha la representación del crucigrama en un archivo de texto

Una vez ya tenemos la plantilla de la cuadrícula escrita le damos la cuadrícula al script y el programa se encarga de crearnos la plantilla en EJJSON [6].

Ya con el archivo en EJJSON creado, tiene que escribir la solución del crucigrama en la cuadrícula, que es simplemente rellenar donde ponga <SOLUTION> el carácter solución de la celda correspondiente. Después en la sección `clues` tiene que rellenar las pistas horizontales y verticales.

Es importante mencionar que antes de que importe el EJON a MongoDB tiene que poner la parte de la hora a cero 00:00:00.000Z, si no lo hace el front no funcionará:

```
"date": { "$date": "2024-06-22T00:00:00.000Z" }
...
```

Una vez que haya hecho las modificaciones pertinentes a la plantilla ya puede importar el EJJSON en la instancia de MongoDB.

Para ejecutar el script por favor consulte el README del repositorio [7] o el Anexo 7 de este documento.

5. Tecnologías

5.1. Express

Para que la aplicación sea interactiva es imprescindible comunicarnos con un servicio que nos diga en el estado de la aplicación. Vamos a utilizar la

arquitectura cliente servidor desarrollando una API que nos permita interactuar con nuestra BBDD e implementar lógica de negocio.

En vez de utilizar los métodos nativos de node para crear un servidor http, utilizaremos la librería de npm **express** que nos ofrece una API más amigable para el desarrollador.

Se han implementado los siguientes endpoints:

```
1 GET    /crosswords
2 GET    /crosswords/AAAA-MM-DD
3 POST   /crosswords/AAAA-MM-DD/solution
4 POST   /hint
```

5.1.1. Listar crucigramas

Para listar todos los crucigramas que hay en la BBDD hay que hacer una llamada HTTP de tipo GET al endpoint `/crosswords`.

Si tiene algún cliente HTTP como `curl` puede ejecutar el siguiente comando:

```
curl localhost:3000/crosswords
```

La petición devuelve un JSON con la siguiente forma:

```
[
  {
    "date": "2024-06-24T00:00:00.000Z",
    "numRows": 6,
    "numCols": 6,
    "numWords": 12,
    "numWordsAcross": 6,
    "numWordsDown": 6
  },
  ...
]
```

5.1.2. Obtener detalles de un crucigrama

Para consultar una crucigrama por su fecha de publicación simplemente tiene que hacer una llamada tipo GET al endpoint `/crosswords/AAAA-MM-DD`, donde las 4 A es el año, las 2 M es el mes con dos dígitos (enero sería 01 no 1) y la DD es el día con dos dígitos (el 1 sería el 01).

Si tiene algún cliente de HTTP puede ejecutar le siguiente comando:

```
curl localhost:3000/crosswords/2024-06-24
```

Devuelve la siguiente respuesta:

```
{
  "xword": {
    "_id": "667ebd00e6de693fc48b98a7",
    "date": "2024-06-24T00:00:00.000Z",
    "numRows": 6,
    "numCols": 6,
    "numWords": 12,
    "numWordsAcross": 6,
    "numWordsDown": 6,
    "clues": {
      "across": [...],
      "down": [...]
    },
    "crossword": [...]
  }
}
```

5.1.3. Comprobar solución del crucigrama

Para comprobar si el crucigrama que ha hecho el usuario es correcto, el usuario envía su solución al servidor al endpoint de tipo POST `/crosswords/AAAA-MM-DD/soluci`.

Para comprobar nuestra solución tenemos que enviar en el cuerpo de la petición un JSON con este esquema:

```
{
  "solution": <formato solución>
}
```

Se tiene que escribir una palabra formada por la concatenación de los caracteres solución de las celdas. El carácter que representa a un bloque es un punto.

Si tenemos el crucigrama solucionado:

```
CBD -----> CBD
B.. -----> CBD, B..
D.. -----> CDB, B.., D..
```

Solución: CBDB..D..

Si tiene un cliente HTTP puede enviar el siguiente comando al servidor.
Cuidado

```
curl -X POST localhost:3000/crosswords/2024-06-24/solution
-H "Content-Type: application/json; charset=utf-8"
-d '{ "solution": "CBDB..D.." }'
```

Si soluciona el crucigrama obtiene la siguiente respuesta:

```
{
  "message": "¡Felicidades!, has solucionado el crucigrama"
}
```

Si se ha equivocado obtiene el siguiente mensaje:

```
{
  "type": "wrong-answer",
  "title": "Solución incorrecta",
  "detail": "Parece que te has equivocado en alguna palabra.
  Deberías revisar las pistas e intentarlo de nuevo",
}
```

5.1.4. Buscar palabras en el diccionario

Para buscar palabras en el diccionario se hace una llamada HTTP de tipo POST al endpoint `/hint` con el siguiente esquema JSON:

```
{
  "from": 0
  "word": "??aco"
}
```

Puede poner los siguientes caracteres en la búsqueda:

- Cualquier letra de la *a* a la *z*, tildes incluidas.
- `?` es un comodín, cualquier letra coincide con este símbolo
- `*` es un símbolo que coincide con una secuencia de cero o más caracteres cualesquiera.

Puede mezclar estos caracteres para hacer búsquedas más complejas:

- La búsqueda **??aco** encuentra registros que terminen en **aco** (ábaco, ataco, braco, etc...)
- La búsqueda **ab*** encuentra palabras que empiecen por ab y terminen en cualquier caracter (aborrascaban, aborrascado, aborrascais, etc...).

Elasticsearch devuelven paginados los resultados, cada página contiene 10 elementos. Si quiere obtener más resultados tiene que cambiar el *offset* o la posición en el cuerpo de la petición.

La siguiente búsqueda tiene 178 resultados:

```
curl -X POST -H "Content-Type: application/json" \
-d '{ "from": 10, "word": "?aco"}' \
http://localhost:3000/hint

{
  "count": 178,
  "matched": [
    "ábaco",
    "abarraco",
    "abellaco",
    "abraco",
    "achaco",
    "afrodisíaco",
    "ajiaco",
    "alaraco",
    "alexifármaco",
    "amáraco"
  ]
}
```

5.2. MongoDB

Para poder comparar las soluciones del usuario y del crucigrama se ha utilizado una agregación de MongoDB para sacar la cadena solución del crucigrama [8](#).

La idea básica con un **reduce** concatenar los caracteres solución de una fila y luego concatenar las cadenas de todas las filas, obteniendo así la cadena solución.

```

1  [
2    {
3      "$match": {
4        "date": "date"
5      }
6    },
7    {
8      "$project": {
9        "solution": {
10         "$reduce": {
11           "input": "$crossword",
12           "initialValue": "",
13           "in": {
14             "$concat": [
15               "$$value",
16               {
17                 "$reduce": {
18                   "input": "$$this",
19                   "initialValue": "",
20                   "in": {
21                     "$concat": ["$$value", "$$this.solution"]
22                   }
23                 }
24               }
25             ]
26           }
27         }
28       }
29     }
30   }
31 ]
32

```

Figura 8: Agregación para obtener solución del crucigrama.

5.3. Elastichsearch

En la etapa de análisis del problema, encontré un video [3] interesante donde mostraban un buscador de palabras en inglés, [4]. Este buscador lo utilizan mucho los crucigramistas de referencia para crear las pistas. Sirviendo de inspiración esta herramienta se ha implementado un buscador de palabras en español.

Es muy conocido ElasticSearch en la industria por ser tecnología muy eficiente para buscar patrones en grandes cantidades de texto. Por lo que en el contexto del proyecto tiene sentido usar ElasticSearch como motor de búsqueda. De hecho el stack de ElasticSearch utiliza Apache Lucene otro buscador motor de búsqueda.

Para hacer las consultas a ElasticSearch se utilizará su API RESTful mediante el cliente de Javascript `@elastic/elasticsearch` en el servidor de Express.

La siguiente consulta busca en el diccionario palabras que tengan en la segunda y en la cuarta posición una a:

```
curl -X GET "localhost:9200/words/_search?pretty" \
-H 'Content-Type: application/json' -d'
{
  "query": {
    "wildcard": {
      "word": {
        "value": "?a?a"
      }
    }
  },
}
```

Utilizando la API de javascript sería de la siguiente forma:

```
elasticSearch.search({
  index: "words",
  query: {
    wildcard: {
      word: "?a?a",
    },
  },
})
```

Podemos ver que la API de javascript es una forma más programática de hacer la consulta. Sin embargo sigue siendo un cliente de la API proporcionando una fachada al programador.

6. Conclusiones

7. Anexo

7.1. Prerequisitos

- NodeJS 20.X.X o superior (<https://nodejs.org/en/>)
- Docker Desktop (Windows, <https://www.docker.com/products/docker-desktop/>), Docker Engine (Linux, <https://docs.docker.com/engine/install/>)

Para arrancar tanto el frontend como el backend se necesita alguna distribución de NodeJS instalada. Para arrancar el backend hay que seguir los siguientes pasos:

Antes de arrancar el back necesita que el driver de mongodb para express se conecte a una instancia en el ordenador, si utiliza docker:

```
> docker run --name xword -p 27017:27017 -d mongo:latest
```

Una vez arrancada la BBDD puede arrancar el backend

Sitúese en la capeta back

```
> cd back
> npm install
> npm build # Crea una carpeta dist en el que se encontrarán
los archivos .js requeridos para ejecutar el servidor de express
```

Después de arrancar mongo ya está en condiciones para arrancar el servidor de express

```
> node dist/index.js
```

Para arrancar el frontend tiene que situarse en la carpeta front y ejecutar

```
> cd front
> npm install
> npm run dev
```

7.2. Script

El script tiene los siguientes parámetros posicionales:

- Número de filas
- Número de columnas
- Ruta del archivo donde se encuentra la cuadrícula con el formato 7 esperado.

Para ejecutar el script primero tendrá que hacer los siguientes pasos:

```
> cd back
> npm install (Opcional si ya ha instalado los node_modules)
> npm build (Se crea un carpeta dist con script.js)
```

Ejemplo de uso:

Windows:

```
> node dist\script.js 11 11 --grid=.\crossword\example.txt
```

Linux:

```
> node ./dist/script.js 11 11 --grid=./crossword/example.txt
```

Referencias

- [1] Shortz, W. (s.f). *Real Rules of the Puzzle*. BarelyBad. <https://barelybad.com/xwdrulesreal.htm>
- [2] Wikipedia (s.f). *Pseudocódigo*. Wikipedia. <https://es.wikipedia.org/wiki/Pseudoc%C3%B3digo>
- [3] WIRED (10 de diciembre de 2018). *How to Create a Crossword Puzzle | WIRED*. [Archivo de Video, OneLook 6m02s]. https://www.youtube.com/watch?v=aAqQnXHd7qk&ab_channel=WIRED
- [4] OneLook (s.f). *OneLook*. <https://onelook.com/>
- [5] Words Up Games (s.f). *Grid Library for Crossword Compiler and Sympathy*. Words Up Games. <https://wordsup.co.uk/grid-libraries.php>
- [6] MongoDB (s.f). *EJSON*. MongoDB. <https://www.mongodb.com/docs/mongodb-shell/reference/ejson/>
- [7] González, P. (26 de Mayo de 2024). *crossword*. crossword. <https://github.com/pgmarc/crossword>
- [8] Duenas, J. (25 de marzo de 2018). *diccionario-espanol-txt*. Diccionario español. <https://github.com/JorgeDuenasLerin/diccionario-espanol-txt>